

TP — Mise en route

Cours Django — IFNTI Sokodé

2 h 30

Ce que vous aurez à la fin

Une application Django qui tourne sur votre machine, dans laquelle vous saisissez des entreprises par l'interface d'administration, et une page web qui en affiche la liste. Le tout dans un dépôt git dont l'historique raconte ce que vous avez fait.

C'est le début du projet que nous construirons pendant tout le semestre : le **suivi des stages de l'IFNTI**. Les entreprises publient des offres, les étudiants candidatent, les stages retenus donnent lieu à une convention et à une évaluation. Aujourd'hui, on pose la première pierre : l'entreprise.

Travaillez par vous-même, mais ne restez pas bloqué plus de dix minutes sans appeler.

1. L'outillage (30 min)

Nous n'utiliserons ni `pip`, ni `virtualenv`, ni `requirements.txt`. Un seul outil, `uv`, fait tout : installer Python, créer l'environnement, gérer les dépendances, lancer les commandes.

1.1 Installer uv

`uv` n'est pas dans les dépôts Ubuntu. Passez par l'installateur officiel :

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Fermez puis rouvrez votre terminal, et vérifiez :

```
uv --version
```

1.2 Créer le projet

```
uv init --no-package suivi_stages && cd suivi_stages
uv python pin 3.14
uv add django==5.2.*
```

`uv init` vous laisse un `main.py` dont un projet Django n'a que faire : supprimez-le, son point d'entrée sera `manage.py`.

Vérifiez que vous avez bien la bonne version des deux côtés :

```
uv run python -V
uv run python -c "import django; print(django.get_version())"
```

Vous devez lire Python **3.14** et Django **5.2**.

1.3 Trois questions avant d'aller plus loin

Votre machine a déjà un Python : `python3 -V` vous le dira, et ce n'est pas 3.14.

a. Qu'a fait `uv python pin 3.14` ? Regardez les fichiers apparus dans le dossier. Et que se serait-il passé si vous l'aviez lancé *avant* d'entrer dans le projet ?

b. Pourquoi le cours impose-t-il une version de Python à toute la classe, alors que Django 5.2 fonctionne aussi bien avec celle de votre Ubuntu ?

c. Que se passerait-il si vous tapiez `python manage.py runserver` au lieu de `uv run manage.py runserver` ? Essayez, plus tard, quand `manage.py` existera.

Gardez vos réponses : nous les reprendrons à la restitution.

2. Le projet Django (30 min)

2.1 Projet et application

Django distingue le **projet** (la configuration : réglages, routage racine) et les **applications** (les morceaux fonctionnels). Créez l'un puis l'autre :

```
uv run django-admin startproject suivi_stages .
uv run manage.py startapp stages
```

Attention au point final de la première commande : sans lui, vous obtenez un dossier de plus.

Lancez le serveur, visitez `http://localhost:8000/`, et laissez-le tourner dans un terminal à part pour tout le TP :

```
uv run manage.py runserver
```

2.2 Lire avant d'écrire

Ouvrez `suivi_stages/settings.py` et `suivi_stages/urls.py`, puis répondez :

a. Quelle ligne de `settings.py` explique que `http://localhost:8000/admin/` réponde déjà quelque chose, alors que vous n'avez rien écrit ?

b. `manage.py` et `django-admin` : à quoi tient la différence ? Lisez `manage.py`, il fait six lignes.

2.3 Une panne à diagnostiquer

Sans rien changer d'autre, lancez :

```
uv run manage.py makemigrations stages
```

Django refuse. Lisez le message, **en entier** — il nomme le problème avec précision.

Réparez. La réparation tient en une ligne, dans un fichier que vous venez d'ouvrir. Notez au passage le nom exact à employer : Django ne devine pas.

3. Le modèle Entreprise (40 min)

3.1 La spécification

Le secrétariat des stages veut enregistrer les entreprises susceptibles d'accueillir un stagiaire. Voici ce qu'il vous dit, dans ses mots :

« On note le nom de l'entreprise, la ville où elle se trouve, son secteur d'activité, et une adresse mail pour la joindre. On ne veut pas se retrouver avec la même entreprise saisie deux fois par deux personnes différentes. Et quand on affiche une entreprise quelque part, on veut la reconnaître tout de suite, sans avoir à cliquer. »

Écrivez le modèle **Entreprise** dans `stages/models.py` à partir de cette demande.

Elle n'est pas complète, et c'est volontaire. Vous allez devoir **trancher**, notamment :

- quel champ, ou quelle combinaison de champs, identifie une entreprise sans ambiguïté ? Une chaîne comme *Ecobank* a une agence à Sokodé et une à Lomé : sont-ce deux entreprises ?
- l'adresse mail : quel type de champ, et pourquoi pas simplement du texte ?
- le secteur d'activité : du texte libre, ou une liste de valeurs imposée ? Qu'est-ce que chaque choix coûte, aujourd'hui et dans trois mois ?
- la dernière phrase du secrétariat ne parle pas d'un champ. De quoi parle-t-elle ?

Vos arbitrages se défendent dans le message de commit, pas en commentaire dans le code. Une phrase par décision suffit, mais elle doit dire *pourquoi*, pas *quoi*.

3.2 Migrer

```
uv run manage.py makemigrations stages
uv run manage.py sqlmigrate stages 0001
uv run manage.py migrate
```

La commande du milieu n'applique rien : elle affiche le SQL que Django s'apprête à exécuter. Lisez-le.

a. Retrouvez dans ce SQL la contrainte qui interdit les doublons. Comment s'appelle-t-elle ?

b. Une colonne n'a d'équivalent dans aucun champ que vous avez écrit. Laquelle, et d'où sort-elle ?

c. Changez `max_length` sur un champ, relancez `makemigrations`, et regardez le fichier engendré dans `stages/migrations/`. Combien de fichiers y a-t-il maintenant ? Que faudrait-il faire pour revenir en arrière ?

4. L'administration (20 min)

Créez votre compte, puis connectez-vous à `http://localhost:8000/admin/` :

```
uv run manage.py createsuperuser
```

Votre modèle n'y apparaît pas : l'admin n'affiche que ce qu'on lui déclare. Déclarez-le dans `stages/admin.py`, puis saisissez cinq entreprises réelles du Togo — celles où vous aimeriez faire votre stage.

Ensuite, améliorez la page de liste : elle doit montrer le nom, la ville et le secteur en colonnes, et offrir une recherche par nom et par ville. Deux attributs suffisent ; la documentation de `ModelAdmin` les donne.

Enfin, testez votre contrainte d'unicité : saisissez une sixième entreprise portant le nom d'une des cinq précédentes.

a. À quel moment l'erreur apparaît-elle : en tapant, en validant, ou plus tard ?

b. Le message affiché est-il compréhensible par la secrétaire qui saisira ces données ?

5. La première page (30 min)

Objectif : `http://localhost:8000/entreprises/` affiche la liste des entreprises saisies.

Il vous faut trois choses :

1. une **vue** dans `stages/views.py`, qui récupère les entreprises et rend un template ;
2. un **template** HTML, que Django doit pouvoir trouver — l'emplacement n'est pas libre, il suit une convention ;
3. deux **routes** : celle de l'application, dans un nouveau `stages/urls.py`, et son inclusion dans `suivi_stages/urls.py`.

Contraintes :

- le `stages/urls.py` déclare `app_name = "stages"` et nomme sa route ;
- la page affiche un message correct quand aucune entreprise n'est enregistrée — vérifiez-le pour de bon, en supprimant temporairement vos entreprises depuis l'admin ;
- pas de CSS aujourd'hui.

Quand la page s'affiche, provoquez l'erreur : déplacez votre template d'un dossier, rechargez, et lisez la page d'erreur de Django jusqu'au bout.

- a. Que liste-t-elle exactement, et en quoi cette liste vous dit-elle où le fichier aurait dû être ?
- b. Cette page très bavarde s'affichera-t-elle aussi le jour où l'application sera en ligne ? De quoi cela dépend-il ?

6. Le dépôt git (15 min)

Votre travail du semestre sera évalué sur *vos* dépôt et *vos* historique. Prenez-le au sérieux dès aujourd'hui.

Vous n'avez pas de dépôt à créer : `uv init` l'a fait, et il a même écrit un `.gitignore`. Ouvrez-le. Il protège l'environnement virtuel et les fichiers compilés de Python, mais **il manque une chose** que vous avez fabriquée cet après-midi et qui n'a rien à faire dans un dépôt. Trouvez-la et ajoutez-la.

```
git add .
git commit -m "..."
```

- a. Quelle est cette chose, et pourquoi ne se committe-t-elle pas, alors qu'elle contient les entreprises que vous venez de saisir ?
- b. Deux fichiers engendrés par `uv` doivent au contraire être commités. Lesquels, et à quoi servent-ils à quelqu'un qui clone votre dépôt ?

Découpez ensuite votre travail en **plusieurs commits** qui racontent l'ordre dans lequel vous avez procédé. Un commit unique nommé « TP1 » ne raconte rien.

À rendre

Avant de partir, poussez votre dépôt sur GitHub et déposez son adresse sur Moodle.

Restitution (20 min)

Nous reprenons ensemble, sans machine. Préparez de quoi répondre à deux d'entre elles.

1. Vous reprenez le projet dans trois mois sur une autre machine. Quelles commandes, dans quel ordre, pour retrouver exactement le même environnement ? Qu'est-ce qui garantit que ce sera *exactement* le même ?

2. Vous avez écrit un modèle, et une table est apparue dans la base. Qui décide de la forme de cette table : votre fichier `models.py`, le fichier de migration, ou la base elle-même ? Justifiez en une phrase.

3. Vous avez rencontré au moins deux erreurs aujourd'hui, provoquées exprès. Pour chacune : qu'est-ce qui, dans le message, vous a mis sur la voie ? Django est bavard — apprendre à lire ce qu'il dit vous fera gagner des heures d'ici la fin du semestre.
